NYC Urban Mobility Data Explorer

Technical Documentation Report

Team Name	Code Sync
Members	Alier Piel · Chol Mach Kuol · Kuol Akech · Abay Tessema
Course	Enterprise Web Development
Submission Date	Thursday, 15 October 2026

1. Problem Framing and Dataset Analysis

Dataset Description

The New York City Taxi and Limousine Commission (TLC) publishes monthly trip records for all yellow taxi rides in New York City. For this project, we used the January 2019 dataset, which contains **7,667,792 raw trip records**. Each record captures the full lifecycle of a single taxi ride: pickup and dropoff timestamps, GPS-based zone IDs, trip distance in miles, an itemised fare breakdown (base fare, tax, tip, tolls, surcharges), payment type, and passenger count.

The dataset was integrated with two supporting files: `taxi_zone_lookup.csv`, which maps the 265 TLC zone IDs to borough names and service zone categories, and `taxi_zones.geojson`, which provides the polygon boundaries for each zone used to render the choropleth map.

Data Challenges

The raw dataset contained several categories of physically and logically impossible records:

Exclusion Reason	Records Removed
Zero distance (cancelled rides / GPS failures)	48,691
Duration over 6 hours (meter left running)	20,463
Dropoff not after pickup (logical impossibility)	6,291
Speed over 80 mph (physically impossible in NYC)	5,921
Fare zero or negative	6,567
Fare below $2.50 TLC flag-drop minimum	391
Pickup outside January 2019 (clock errors)	537
Distance over 100 miles (odometer / GPS error)	31
Fare above $500 (data-entry error)	4
Negative charge component	6
Total excluded	**88,902 (1.159%)**

Every excluded record was logged with its reason code and stored in the `excluded_records` table for full transparency. Beyond exclusions, a further **369,285 records** were kept but flagged or repaired:

Repair / Flag Reason	Records Affected
Unknown dropoff zone (TLC zone IDs 264/265)	154,335
Unknown pickup zone	152,317
Zero passengers — set to NULL	115,074
Undocumented vendor ID	76,380
Total amount ≠ sum of fare components	15,751
Invalid rate code — set to NULL	106

Assumptions Made During Cleaning

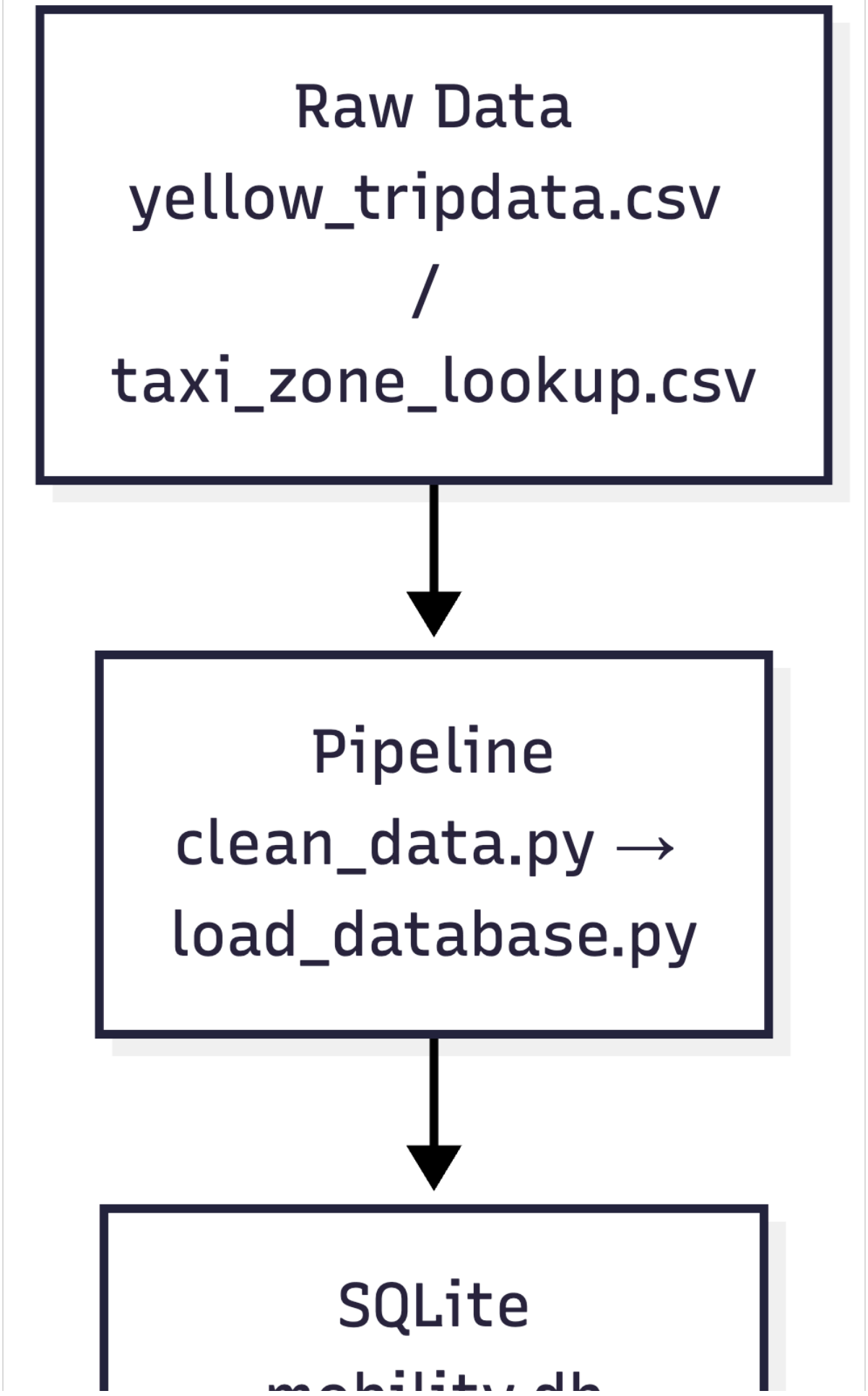
- Any trip with zero or negative duration was removed as logically impossible regardless of distance.
- Speed was calculated as distance divided by duration in hours; anything above 80 mph was excluded as physically impossible on NYC roads.
- The $2.50 threshold for fare exclusion was chosen because it is the TLC-mandated flag-drop fare — no metered ride can cost less.
- "0 passengers" was not treated as an error. The field is frequently left blank by drivers; removing these trips would have eliminated 115,074 otherwise valid rides.
- Trips with unknown zones (264/265) were kept because the trip data itself (fare, distance, duration) is valid — only the geographic attribution is missing.

Unexpected Observation

The most surprising finding was the scale of the unknown zone problem. We expected a small number of GPS failures, but 154,335 dropoff zones and 152,317 pickup zones were unresolvable — nearly 2% of all trips. This influenced our design decision to keep these trips in the database rather than exclude them, and to display "Unknown" explicitly in the dashboard rather than silently dropping the records. It also led us to filter out unknown zones from the top-routes table by default, since the largest single "route" in the raw data was Unknown → Unknown with over 200,000 trips — a statistical artefact, not a real mobility pattern.

2. System Architecture and Design Decisions

Architecture Diagram


Raw Data
yellow_tripdata.csv
/
taxi_zone_lookup.csv
Pipeline
clean_data.py →
load_database.py
SQLite

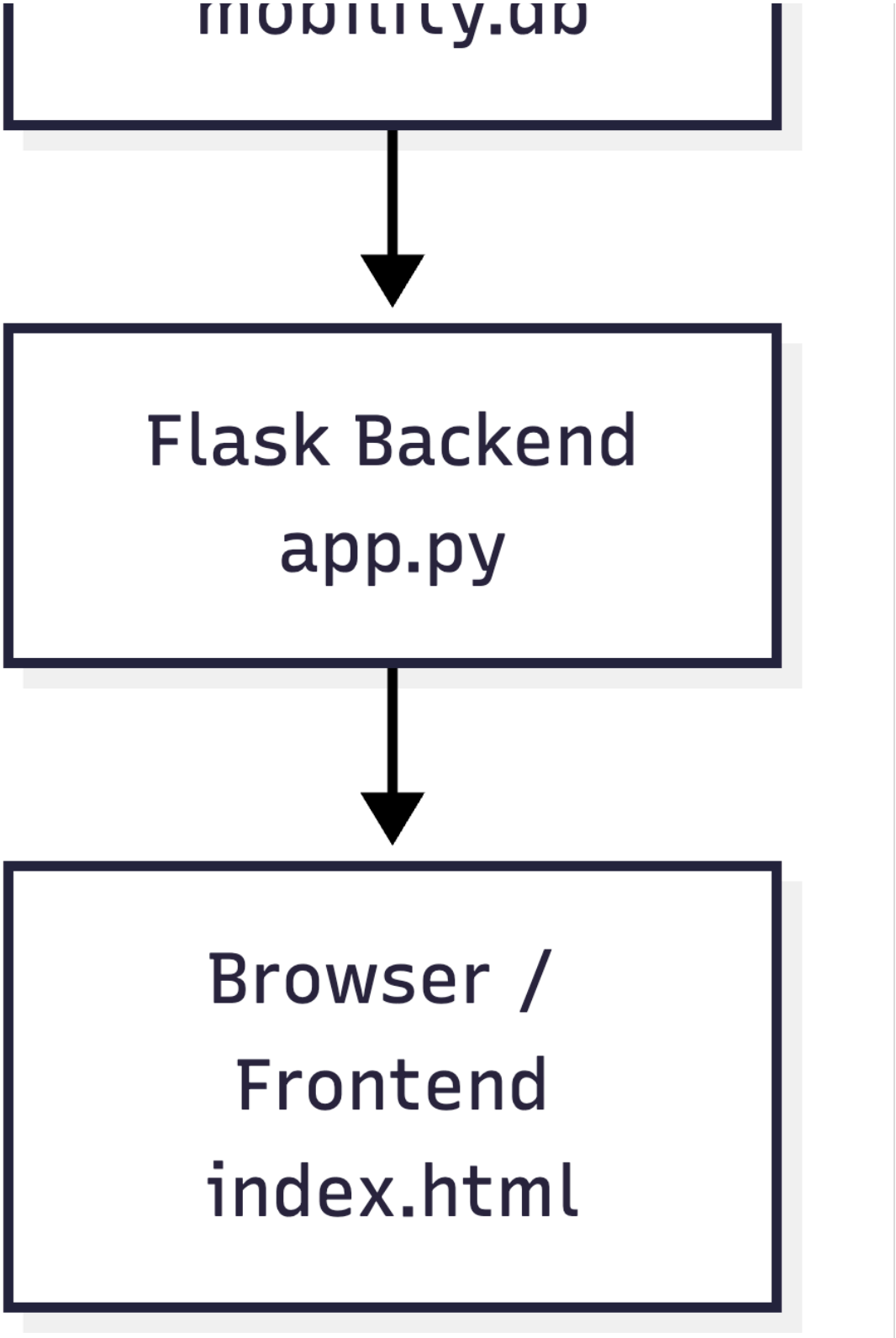


Figure 1 — Three-tier architecture: pipeline → SQLite → Flask API → browser dashboard

System Overview

The application follows a three-tier architecture with a data pipeline feeding the storage layer:

- **Pipeline layer:** Raw Parquet and GeoJSON files are processed by `pipeline/clean_data.py` (cleaning and feature engineering) and

`database/load_database.py` (loading into SQLite). This runs once to build the database.

- **Data layer:** A SQLite database (`data/mobility.db` , 2.82 GB) stores 7,578,890 cleaned trips plus dimension tables, summary tables, and the exclusion log.
- **Backend layer:** A Flask application (`backend/app.py`) exposes 12 REST API endpoints. It reads from the database and returns JSON. It also serves the frontend files, so a single command starts the entire application.
- **Frontend layer:** A browser-based dashboard (`frontend/index.html`) fetches data from the Flask API and renders interactive SVG charts, a choropleth map, a sortable trip table, and KPI cards — all without any frontend framework.

Stack Choices and Justification

Flask was chosen for the backend because the entire team was already working in Python for the pipeline, and Flask's minimal structure meant we could expose API endpoints quickly without learning a new framework.

SQLite was chosen over PostgreSQL or MySQL because the dataset fits in a single file, requires no server process or configuration, and is fast enough for read-only analytical queries. The 2.82 GB database fits comfortably in memory on a modern machine, and SQLite's WAL mode handles concurrent reads from the dashboard without locking.

Vanilla JavaScript with hand-drawn SVG was chosen for the frontend to avoid framework overhead and to demonstrate direct understanding of the browser's rendering model. Every chart — the line charts, heatmap, bar charts, and choropleth map — is drawn by calculating pixel coordinates from data values and writing SVG elements directly to the DOM.

Schema Structure

The database uses a **star schema** normalised to third normal form (3NF):

- **Fact table:** `trips` — one row per cleaned ride, 7,578,890 rows, 25 columns including raw fields and derived features.
- **Dimension tables:** `zones` , `boroughs` , `vendors` , `rate_codes` , `payment_types` — small lookup tables referenced by foreign key from `trips` .
- **Summary tables:** `zone_date_hour_stats` (~100,762 rows) and `route_date_stats` (~321,988 rows) — pre-aggregated totals built by the loader.
- **Transparency tables:** `excluded_records` and `exclusion_reasons` — the full audit log of every removed record.

Trade-offs

SQLite vs PostgreSQL: SQLite does not support concurrent writes, but since the dashboard is entirely read-only after the database is built, this was not a constraint. The

benefit was zero server configuration and a single portable file.

Pre-aggregated summary tables: Adding `zone_date_hour_stats` and `route_date_stats` increases storage by approximately 50 MB but reduces the most common dashboard queries from scanning 7.5 million rows to scanning ~100,000 rows. The API's `source_for()` function automatically selects the summary table when the active filters are compatible with it.

Hand-drawn SVG vs a charting library: Using Chart.js or D3 would have been faster to implement, but writing SVG directly gave us full control over layout, colour, and interaction, and avoided adding a large dependency to a project that is otherwise dependency-free on the frontend.

3. Algorithmic Logic and Data Structures

Implemented by Chol Mach Kuol — `backend/algorithms.py`

Three algorithms were manually implemented without using built-in sorting, ranking, or counting libraries. All three are called directly by `app.py` at runtime.

3.1 top_k(items, k, key)

Problem: The `/api/top-routes` endpoint needs to return the top k routes by trip count from a list of up to 63,000 route records. Sorting the full list on every request is wasteful when only the top 10 are needed.

Approach: A min-heap of fixed size k is maintained. For each item, if its score exceeds the smallest item currently in the heap, it replaces it. At the end, the heap contains exactly the k largest items.

```
heap = []
for each item in items:
    val = key(item)
    if size of heap < k:
        push (val, item) onto heap
    else if val > minimum value in heap:
        replace minimum in heap with (val, item)
return heap sorted by val descending
```

Time complexity: O(n log k) time, O(k) space. When k = 10 and n = 63,000, this is significantly faster than O(n log n) full sort.

3.2 count_by(items)

Problem: The zone detail panel needs to count how many trips went to each dropoff zone from a list of trip records fetched for a single pickup zone.

Approach: A single pass over the list, incrementing a dictionary counter for each value seen. No sorting, no library functions.

```
counts = empty dictionary
for each item in items:
    if item is in counts:
        counts[item] = counts[item] + 1
    else:
        counts[item] = 1
return counts
```

Time complexity: O(n) time, O(u) space where u is the number of unique values (at most 265 zones).

3.3 quickselect_median(values)

Problem: Fare distributions are heavily skewed by outliers — airport rides to JFK can cost $200 or more. The arithmetic mean misrepresents the typical fare. The median is more robust, but sorting the full list costs O(n log n).

Approach: Quickselect — a partition-based selection algorithm derived from quicksort. A pivot is chosen from the middle of the list, the list is partitioned into elements smaller and larger than the pivot, and the algorithm recurses only on the partition that contains the target index.

```
function quickselect(lst, k):
    pick pivot = lst[middle index]
    i = left, j = right
    while i <= j:
        advance i while lst[i] < pivot
        retreat j while lst[j] > pivot
        if i <= j: swap lst[i] and lst[j]; i++; j--
    if k <= j: recurse on left half
    else if k >= i: recurse on right half
    else: return lst[k]

if n is odd:
    median = quickselect(lst, n // 2)
if n is even:
    median = (quickselect(lst, n//2 - 1) + quickselect(lst, n//2)) / 2
```

Time complexity: O(n) average time, O(1) extra space (in-place partition). Worst case is O(n²) if the pivot is always the minimum or maximum, but this is rare in practice with a middle-index pivot on real fare data.

4. Insights and Interpretation

Insight 1 — The slower the city, the more each mile costs

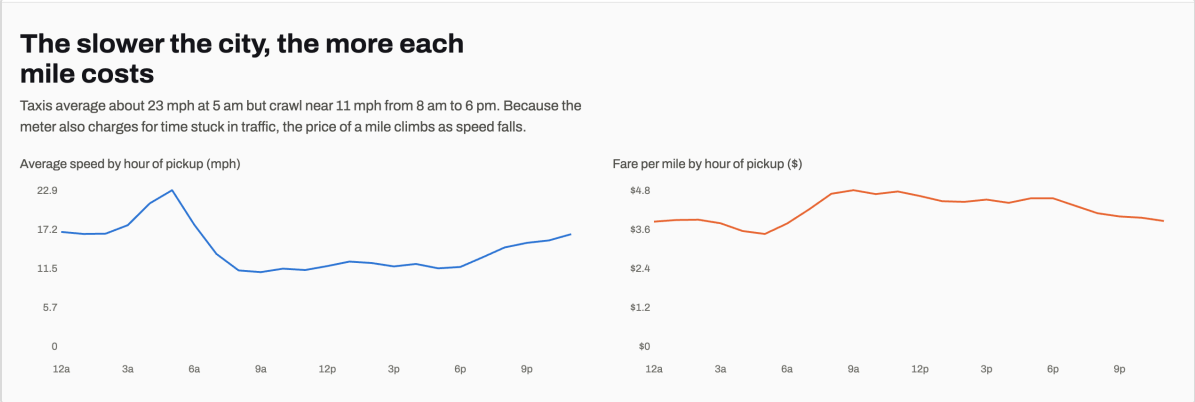


Figure 2 — Average speed (mph) and fare per mile ($) by hour of day across January 2019

Taxis average approximately 23 mph at 5 am when the streets are empty, but slow to under 11 mph between 8 am and 6 pm during peak congestion. Because the TLC meter charges for both distance travelled and time elapsed, the fare per mile rises as speed falls — from around $3.50 per mile at night to nearly $5.00 per mile during the morning rush hour. This means a passenger travelling the same physical distance pays roughly **40% more during rush hour than at night**, purely because of traffic. The relationship is consistent across the entire month, confirming that congestion is the primary driver of fare variability in New York City, not distance.

Insight 2 — Thursday and Friday evenings are the city's peak

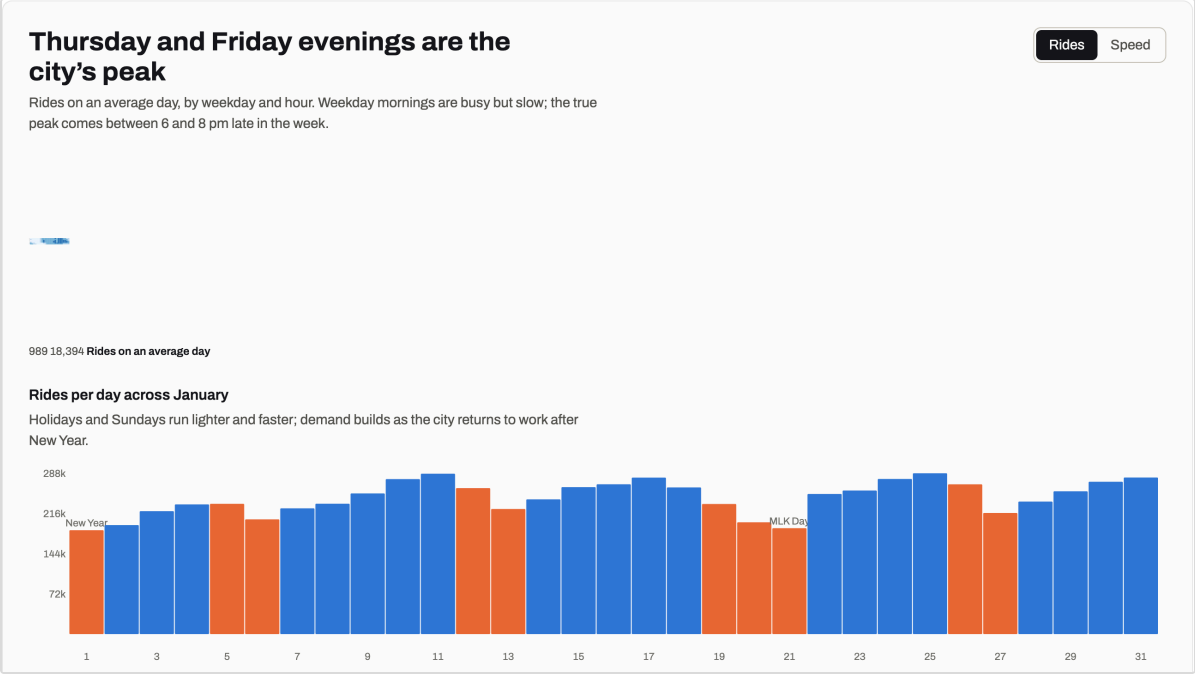
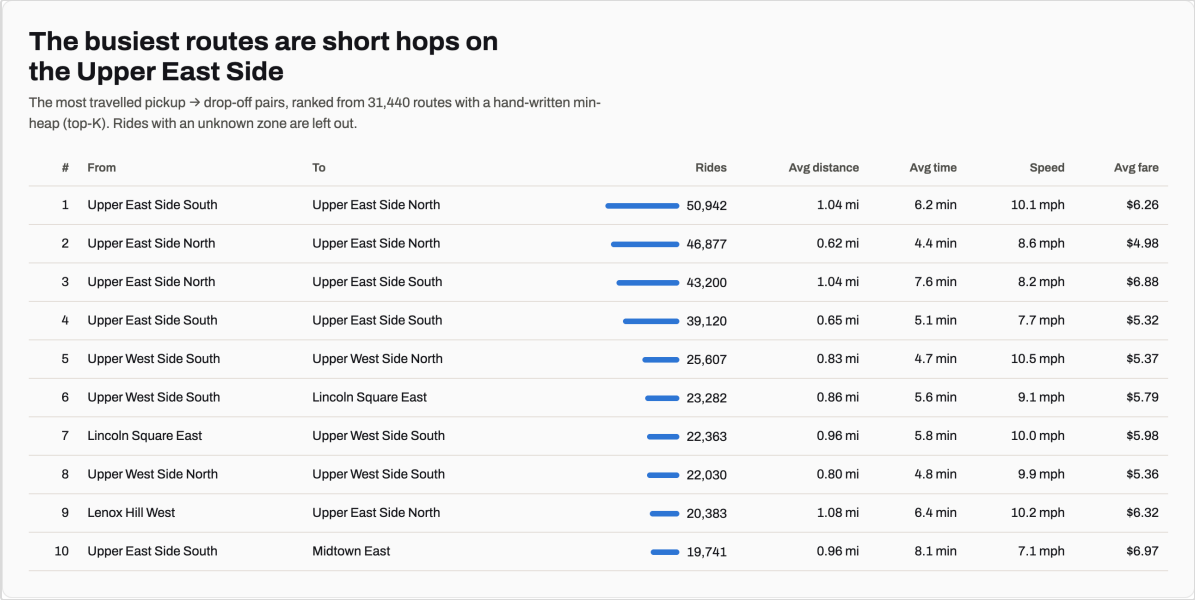


Figure 3 — Trip volume heatmap: weekday (rows) × hour of day (columns)

The weekday-by-hour heatmap shows that demand is not uniform across the week. Weekday mornings (7–9 am) are consistently busy, but the true peak — both in trip volume and in congestion — occurs between **6 pm and 8 pm on Thursday and Friday evenings**. Saturday shows a different pattern: demand is spread more evenly across the afternoon and evening rather than concentrated in two rush-hour spikes. Sunday is the quietest day across all hours. This pattern reflects the structure of New York City's working week: the end-of-week evening surge is driven by commuters, social trips, and the start of the weekend, all coinciding on the same streets at the same time.

Insight 3 — Nine in ten rides start in Manhattan, and the busiest routes are short hops



#	From	To	Rides	Avg distance	Avg time	Speed	Avg fare
1	Upper East Side South	Upper East Side North	50,942	1.04 mi	6.2 min	10.1 mph	$6.26
2	Upper East Side North	Upper East Side North	46,877	0.62 mi	4.4 min	8.6 mph	$4.98
3	Upper East Side North	Upper East Side South	43,200	1.04 mi	7.6 min	8.2 mph	$6.88
4	Upper East Side South	Upper East Side South	39,120	0.65 mi	5.1 min	7.7 mph	$5.32
5	Upper West Side South	Upper West Side North	25,607	0.83 mi	4.7 min	10.5 mph	$5.37
6	Upper West Side South	Lincoln Square East	23,282	0.86 mi	5.6 min	9.1 mph	$5.79
7	Lincoln Square East	Upper West Side South	22,363	0.96 mi	5.8 min	10.0 mph	$5.98
8	Upper West Side North	Upper West Side South	22,030	0.80 mi	4.8 min	9.9 mph	$5.36
9	Lenox Hill West	Upper East Side North	20,383	1.08 mi	6.4 min	10.2 mph	$6.32
10	Upper East Side South	Midtown East	19,741	0.96 mi	8.1 min	7.1 mph	$6.97

Figure 4 — Choropleth map of pickup density by zone, and top 10 routes by trip count

The choropleth map shows that pickup activity is overwhelmingly concentrated in Manhattan, which accounts for the vast majority of all 7.5 million trips. The top routes table reinforces this: the single busiest route in January 2019 was **Upper East Side South → Upper East Side North**, with 50,942 trips averaging just 1.04 miles and 6.2 minutes. All ten of the busiest routes are short intra-Manhattan hops under 1.1 miles. This reveals that yellow taxis in January 2019 functioned primarily as a short-distance urban transit option for Manhattan residents rather than a cross-borough or airport service — a finding with direct implications for how the city should think about taxi infrastructure and congestion pricing.

5. Reflection and Future Work

Technical Challenges

Alier Piel — Backend

The most challenging part of the backend was designing the query routing logic in `source_for()`. The pre-aggregated summary tables are much faster, but they only support a subset of filters. Writing the logic to detect which filters are active and automatically choose the right table — without the API caller needing to know — required careful thought about which columns exist in each table and how to avoid returning wrong results when a filter is not supported by the summary table.

Chol Mach Kuol — Algorithms

Building the algorithms without standard libraries forced careful thinking about what each function actually does rather than relying on a black box. The hardest part was getting quickselect correct for even-length lists — the median requires two separate calls to the partition function with different target indices, and getting the index arithmetic right for both odd and even cases took several iterations. The min-heap for `top_k` also required careful handling of tie-breaking to ensure consistent ordering when two routes have the same trip count.

Kuol Akech — Frontend

The most difficult part of the frontend was building the choropleth map entirely in SVG without a mapping library. Projecting GeoJSON polygon coordinates onto a fixed SVG canvas required implementing a manual bounding-box projection — finding the min/max longitude and latitude across all zone polygons, then scaling each coordinate to fit the canvas dimensions. Getting the zoom and pan interactions to feel natural without a library took significant iteration.

Abay Tessema — Pipeline

The main challenge in the pipeline was handling the variety of data quality issues without over-removing valid records. The decision of where to draw the line — exclude vs repair vs flag — required reading the TLC data dictionary carefully and making judgment calls about which anomalies were genuine errors versus quirks of how drivers use the system. The 115,074 zero-passenger trips were the clearest example: a naive approach would exclude them, but they are distributed uniformly across all hours and vendors, which strongly suggests a data-entry habit rather than an error.

Future Improvements

- **Deployment:** The application could be deployed to AWS using EC2 for the Flask backend and S3 for static frontend assets, making the dashboard publicly accessible

without requiring local setup.

- **More months of data:** Adding the full 2019 dataset (~84 million trips) would allow seasonal analysis — comparing January's cold-weather patterns against summer and autumn.
- **Real-time data:** The TLC publishes trip data with a one-month lag. A scheduled pipeline could automatically fetch and load each new month's data as it becomes available.
- **Predictive features:** With a full year of data, a simple time-series model could predict demand by zone and hour, which would be useful for both passengers and drivers planning their day.

Data source: NYC Taxi & Limousine Commission, Yellow Taxi Trip Records, January 2019. | Built with Flask, SQLite, and hand-drawn SVG.